



VERTEX

Programming Language Reference

Language rules, syntax, standard helpers, VCL, ComPort, Vertex IDE, and the vertexc compiler

Version 1.0

Companion to Vertex IDE and the Vertex Component Library

Contents

1. Introduction
2. Program Structure
3. Lexical Rules
4. Types
5. Declarations
6. Expressions and Operators
7. Statements
8. Procedures and Functions
9. Records and Classes
10. Modules and Import
11. The Asm Escape
12. Vertex Component Library (VCL)
13. ComPort (Serial Communication)
14. Vertex IDE
15. Compiler (vertexc) and Build Tool
16. Complete Examples
17. Keyword and Token Reference
18. Error Messages and Common Pitfalls

1. Introduction

Vertex is a structured, imperative language designed for clear GUI and systems programming. Source files use the .vtx extension. The vertexc compiler parses Vertex, expands textual imports, and emits C++ which is then compiled with a standard C++ toolchain (for example g++).

Design goals

- Readable Pascal-like syntax with Run / Stop blocks instead of begin / end.
- Strong emphasis on explicit structure: Enter / Exit for the main program.
- Native Windows GUI through the Vertex Component Library (VCL).
- Seamless escape to C++ via Asm when the runtime or Win32 API needs it.
- Bidirectional form designer in Vertex IDE: code and visual layout stay in sync.
- Optional serial (ComPort) helpers for device and laboratory style applications.

Hello World

```
Enter
  Print("Hello, Vertex!");
Exit.
```

GUI Hello (with VCL)

```
Import "vcl.vtx";

Proc OnBtn(h: HWND);
Run
  ShowMessage("Hello from Vertex", "Greeting");
Stop;

Enter
  Window(400, 200, 100, 100);
  SetWindowTitle("Hello");
  btn <- Button(0, 120, 30, 140, 80);
  SetText(btn, "Click me");
  OnClick(@OnBtn);
  RunApp();
Exit.
```

2. Program Structure

A Vertex compilation unit is ordered. Declarations come first, then the main body.

Order of sections

```
Import ...;           { optional, may appear several times }
Const ...;           { optional }
Type ...;            { optional }
Var ...;             { optional }
Func / Proc ...;     { optional, may appear several times }
Enter
{ main statements }
Exit.
```

The keywords Enter and Exit mark the main program. The body between them is executed at startup. Exit ends the unit. A trailing period after Exit is accepted in the classic style (Exit.).

3. Lexical Rules

Comments

- Brace comments: { this is a comment } (may span multiple lines)
- Line comments: // to end of line

Comments may not be nested. Import directives inside comments are ignored.

Identifiers

Identifiers start with a letter or underscore, followed by letters, digits, or underscores. Vertex keywords are case-sensitive and use the capitalised forms listed in chapter 17 (for example If, Then, Else, not if, then, else).

Literals

Kind	Examples	Notes
Integer	0, 42, 1001	Decimal only
Real	3.14, 0.5	Dot decimal
String	"Hello"	Double quotes
Character	via Char / Chr	Often obtained with Chr(code)
Boolean	True, False	Keywords

Important: avoid putting a double-quote character inside an Asm string. Build strings with variables when quotes would be required inside Asm.

Encoding

Source must be UTF-8 or plain ASCII. UTF-16 and UTF-32 files are rejected with a clear error.

4. Types

Built-in scalar types

Vertex type	Emitted C++	Description
Integer	int	Signed integer
Real	double	Floating point
Boolean	bool	True / False
String	std::string	Dynamic text
Char	char	Single character
Byte	unsigned char	8-bit value
PChar	const char*	C string pointer

Win32 types such as HWND, UINT, WPARAM, LPARAM, LRESULT, HINSTANCE, and MSG pass through unchanged when windows.h is imported. They are treated as ordinary type names.

Arrays

Arrays use inclusive bounds. Array[0..511] Of Char has 512 elements.

```
Var
  buffer: Array[0..511] Of Char;
  nums: Array[1..10] Of Integer;
```

The compiler emits C arrays with the correct size, for example char buffer[512];

Pointers

```
Type
  PInteger = ^Integer;
```

Use @ to take an address, and ^ to dereference (emitted as C *).

Procedure and function types

```
Type
  TNotifyEvent = Proc(hCtrl: HWND);
  TIntFunc = Func(x: Integer): Integer;
```

These become C++ function pointers. Assign with @HandlerName.

5. Declarations

Constants

```
Const
  MaxItems = 100;
  AppName = "VertexApp";
```

Variables

```
Var
  count: Integer;
  title: String;
  MainWindow: HWND;
```

Type aliases and structured types

```
Type
  TPoint = Record
    x: Integer;
    y: Integer;
  End;
  TNotifyEvent = Proc(hCtrl: HWND);
```

6. Expressions and Operators

Assignment

Assignment uses the arrow token `<-` (less-than followed by hyphen). The equals sign `=` is comparison, not assignment.

```
count <- 0;
title <- "Hello";
flag <- True;
```

Arithmetic

Operator	Meaning	C++ emit
+	Addition	+
-	Subtraction / unary minus	-
*	Multiplication	*
/	Division	/
Div	Integer division	/
Mod	Modulo	%
Shl	Shift left	<<
Shr	Shift right	>>

Comparison

Operator	Meaning	C++ emit
=	Equal	==
<>	Not equal	!=
<	Less than	<
>	Greater than	>
<=	Less or equal	<=
>=	Greater or equal	>=

Logical and bitwise

Operator	Meaning	C++ emit
And	Bitwise / logical and	&
Or	Bitwise / logical or	
Xor	Bitwise xor	^
Not	Logical not	!

And / Or emit bitwise operators. For window style flags this is correct (`WS_CHILD` Or `WS_VISIBLE`). For pure boolean tests, prefer comparisons against 0 or True / False.

Other operators

- `@expr` address of (emits `&`)
- `ptr^` dereference (emits `(*ptr)`)
- `a[i]` array index
- `r.field` record or struct field
- `SizeOf(T)` size in bytes

7. Statements

Blocks: Run / Stop

A block is introduced with Run and closed with Stop. Blocks are used as procedure bodies, Then / Else bodies, loop bodies, and similar.

```
Run
  x <- 1;
  y <- 2;
Stop;
```

If / Then / Else

A semicolon ends a statement, but a semicolon is optional immediately before Else, because Else belongs to the enclosing If.

```
If x = 0 Then
  y <- 1
Else
  y <- 2;

If x > 0 Then
Run
  Print("positive");
  y <- x;
Stop
Else
Run
  Print("not positive");
  y <- 0;
Stop;
```

Else If chains are written as Else followed by another If. The emitter always braces C++ if/else bodies to avoid dangling-else problems.

Loops

```
While n > 0 Do
Run
  n <- n - 1;
Stop;

For i <- 1 To 10 Do
  Print(i);

For i <- 10 Downto 1 Do
Run
  Print(i);
Stop;

Repeat
  n <- n + 1;
Until n >= 10;
```

Break leaves the nearest loop. Continue starts the next iteration.

Print and Input

```
Print("Value = ", x);
Input(a);    { read into variable a }
Input();     { wait for Enter }
```

FloatToStr formats whole numbers without a trailing decimal fraction when the value is an integer.

New and Dispose allocate and release (toward C++ new / delete).

8. Procedures and Functions

Proc has no return value. Func returns a value. Assign to the function name (Pascal style); the compiler turns that into a local `_result` variable.

```
Func Add(a: Integer; b: Integer): Integer;
Run
  Add <- a + b;
Stop;

Proc ShowSum(a: Integer; b: Integer);
Var
  s: Integer;
Run
  s <- Add(a, b);
  Print(s);
Stop;
```

9. Records and Classes

```
Type
  TPoint = Record
    x: Integer;
    y: Integer;
  End;
```

Classes support fields, methods, constructors, and destructors. Visibility keywords (Public, Private, Virtual, Override) are accepted; layout currently treats members as public in the C++ emission.

10. Modules and Import

```
Import "vcl.vtx";      { textual include of another Vertex file }
Import <windows.h>;    { system header, kept as C++ #include }
Import <cstring>;
```

When the path ends with `.vtx`, the preprocessor recursively expands that file (depth limit 32). Non-`.vtx` quoted imports and angle-bracket imports become C++ `#include` lines.

11. The Asm Escape

Asm injects a raw C++ statement into the generated code. It is not CPU assembly. Use it for APIs or expressions that Vertex does not express yet.

```
Var
  result: Integer;
Run
  Asm "result = (int)strlen(s.c_str());";
  Length <- result;
Stop;
```

Rules

- The Asm string must not contain a double-quote character.
- Prefer building C strings via Vertex String variables and `.c_str()` inside Asm.
- End the injected C++ with a semicolon when the statement requires one.
- Local Vertex variables are visible by name inside the injected fragment.

12. Vertex Component Library (VCL)

VCL (vcl.vtx) is a Vertex library that wraps Win32 window and control creation. Import it at the top of a GUI program.

```
Import "vcl.vtx";
```

Window lifecycle

API	Role
Window(w, h, x, y)	Create and show the main window
SetWindowTitle(text)	Set main window caption
SetFormColor(color)	Solid form background (ColorRGB)
RunApp()	Message loop until the window closes

Controls

API	Returns	Role
Button(parent, w, h, x, y)	HWND	Push button (owner-draw when colored)
Edit(parent, w, h, x, y)	HWND	Single-line edit
Label(parent, w, h, x, y)	HWND	Static text
Memo(parent, w, h, x, y)	HWND	Multiline edit
CheckBox(parent, w, h, x, y)	HWND	Auto checkbox
Radio(parent, w, h, x, y)	HWND	Auto radio button
ListBox(parent, w, h, x, y)	HWND	List box
ComboBox(parent, w, h, x, y)	HWND	Dropdown combo
GroupBox(parent, w, h, x, y)	HWND	Group frame
Panel(parent, w, h, x, y)	HWND	Simple panel

Pass 0 as parent to place the control on MainWindow. Each control receives a unique numeric ID used by click dispatch.

Control operations

API	Role
SetText(h, text) / GetText(h)	Caption or text
SetBounds(h, x, y, w, h)	Move and resize
ShowCtrl / HideCtrl	Visibility
EnableCtrl / DisableCtrl	Enabled state
ShowMessage(text, title)	Modal information box
OnClick(handler)	Register a TNotifyEvent click handler
SetChecked / GetChecked	CheckBox and Radio state
AddItem / ClearItems	ListBox and ComboBox items
GetItemCount / GetItemIndex / SetItemIndex	List selection
ColorRGB / SetFormColor / SetBackColor / SetCtrlTextColor	Colors

Forms use SetFormColor with WM_ERASEBKGD. Labels, edits, and list boxes use WM_CTLCOLOR* messages. Push buttons that receive SetBackColor are switched to owner-draw and painted in WM_DRAWITEM so modern visual styles do not ignore the color.

Helpers

Length, Ord, Chr, Trunc, Round, FloatToStr, StrToFloat, IntToStr, StrToInt are provided by VCL. FloatToStr avoids trailing .0000 style output when the value is a whole number.

Implementation notes

- VCL uses ANSI Win32 APIs (CreateWindowA, SetWindowTextA, and similar).
- Control IDs are passed as the HMENU argument at creation time (correct on 64-bit).
- The window class is registered once per process.
- WndProc uses sequential If checks rather than deep Else chains for robustness.

13. ComPort (Serial Communication)

VCL includes a minimal Win32 serial port API. Handles are stored as Integer. Value 0 means closed or invalid. Default framing is 8 data bits, no parity, 1 stop bit.

API	Role
ComOpen(port, baud)	Open port at baud rate
ComClose(h)	Close the port
ComWrite(h, data)	Write a String; returns bytes written
ComRead(h, maxLen)	Read up to maxLen bytes into a String
ComBytesAvailable(h)	Number of bytes waiting in the input queue

Port name examples: COM1, COM3. For COM10 and above, Windows often requires the \\.\COMn form.

```
hPort <- ComOpen("COM3", 9600);
If hPort = 0 Then
  ShowMessage("Open failed", "ComPort")
Else
  Run
  n <- ComWrite(hPort, "AT");
  data <- ComRead(hPort, 64);
  ComClose(hPort);
Stop;
```

14. Vertex IDE

Vertex IDE is a Delphi-inspired environment with a code editor, form designer, component palette, property panel, compile/run pipeline, and themes (dark, light, monokai).

Designer features

- Drag and drop controls from the palette onto the form (including ComPort).
- Grid (about 15 px), resize handles for controls and the form.
- Property editor: Name, Caption, Left, Top, Width, Height, Color (scrollable list with swatches).
- Code and Designer sync: Generate Code writes Vertex; Sync rebuilds the form from code.
- Manual resize and move update the source when markers are present.
- Designer-only controls are kept when Sync runs before Generate so they do not vanish.
- Drag and resize update only the selected control for responsiveness on large forms.
- Help menu can open documentation.pdf.
- Build is separated into vertex_build.py which returns structured JSON errors to the IDE.

Form Designer overview

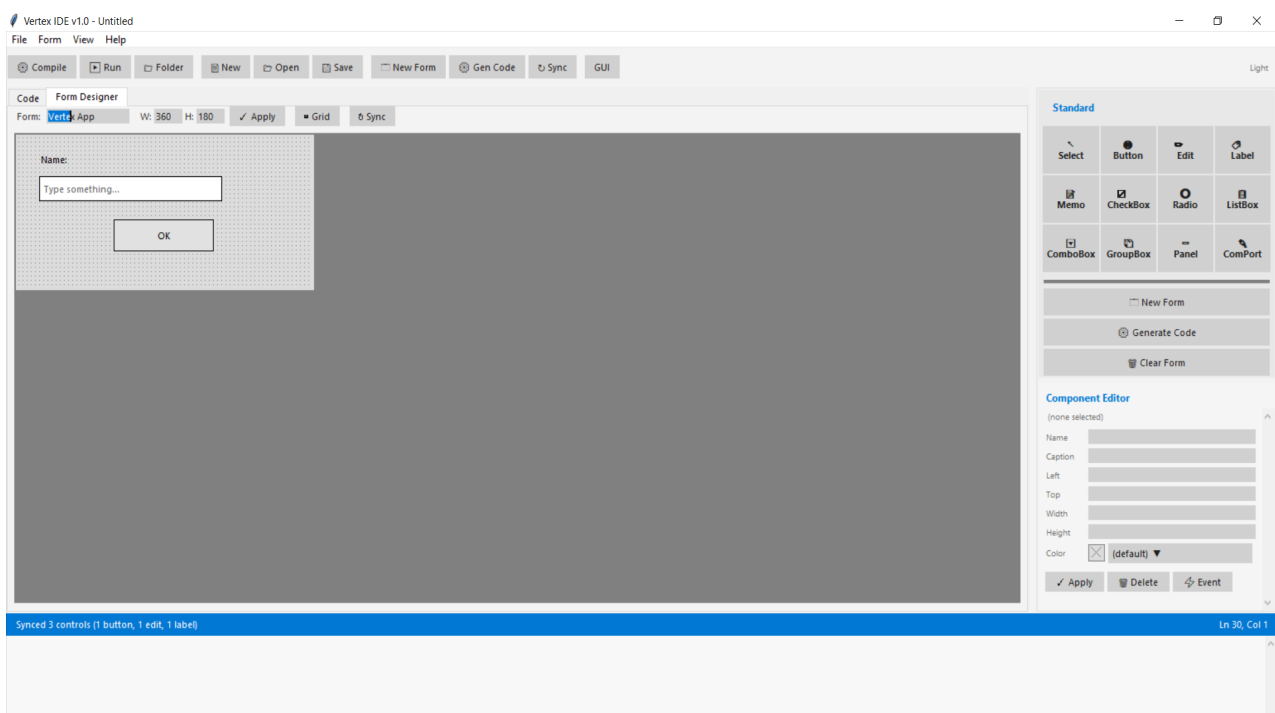


Figure: Vertex IDE Form Designer with palette, form surface, and component editor.

15. Compiler (vertexc) and Build Tool

Build the compiler

```
gcc -O2 -std=c99 vertexc.c -o vertexc.exe
```

Compile a Vertex program

```
vertexc.exe MyApp.vtx  
g++ -O2 -std=c++17 output.cpp -o MyApp.exe -mwindows -static -luser32 -lgdi32
```

vertexc always writes output.cpp. GUI programs need -mwindows and the user32 / gdi32 libraries. Vertex IDE automates this pipeline through vertex_build.py.

```
python vertex_build.py MyApp.vtx --mode auto --vertexc vertexc.exe --gpp g++
```

Pipeline

- Preprocess: expand Import "file.vtx" recursively.
- Lex and parse into an AST.
- Emit C++ (types, functions, main).
- Host C++ compiler links the final executable.

When windows.h is imported, the emitter adds small wrappers so SetWindowTextA and MessageBoxA accept std::string as well as const char*.

16. Complete Examples

Console: sum of 1 to N

```
Var  
  n: Integer;  
  i: Integer;  
  sum: Integer;  
Enter  
  n <- 10;  
  sum <- 0;  
  For i <- 1 To n Do  
    sum <- sum + i;  
  Print("Sum = ", sum);  
Exit.
```

Console: Input

```
Enter ConsoleCalc;  
Var  
  a: Integer;  
  b: Integer;  
  sum: Integer;  
Run  
  Print("Enter first number: ");  
  Input(a);  
  Print("Enter second number: ");  
  Input(b);  
  sum <- a + b;  
  Print("Sum: ", sum);  
  Print("Press Enter to exit...");  
  Input();  
Stop  
Exit.
```

GUI: colored button

```
Import "vcl.vtx";  
Var  
  btn: HWND;  
Proc OnAny(h: HWND);  
Run  
  ShowMessage("Clicked", "Event");  
Stop;  
Enter  
  Window(400, 240, 120, 120);  
  btn <- Button(0, 120, 36, 40, 40);  
  SetText(btn, "OK");  
  SetBackColor(btn, ColorRGB(70, 130, 180));  
  SetCtrlTextColor(btn, ColorRGB(255, 255, 255));  
  OnClick(@OnAny);
```

```
RunApp();  
Exit.
```

ComPort open and write

```
Import "vcl.vtx";  
Var  
  hPort: Integer;  
  n: Integer;  
Enter  
  hPort <- ComOpen("COM3", 9600);  
  If hPort <> 0 Then  
    Run  
      n <- ComWrite(hPort, "AT");  
      ComClose(hPort);  
    Stop;  
  Exit.
```

17. Keyword and Token Reference

Keywords (case-sensitive)

Category	Keywords
Structure	Enter Exit Run Stop Import
Declarations	Const Type Var Func Proc Class Record Extends
Visibility	Private Public Virtual Override
Control flow	If Then Else For To Downto Do While Repeat Until
Other flow	With Attempt Recover Break Continue
Memory	New Dispose Absolute
IO and util	Print Input SizeOf OffsetOf Asm
Types / values	Array Of End True False
Operators (words)	And Or Xor Not Div Mod Shl Shr
Methods	Constructor Destructor

Special tokens

Token	Role
<-	Assignment
=	Equality comparison (also type binding in declarations)
<>	Inequality
@	Address-of
^	Pointer dereference / pointer type
..	Array range
;	Statement separator (optional before Else)

18. Error Messages and Common Pitfalls

expected ';' (found Else)

Older compiler builds required a semicolon even before Else. Current vertexc allows the Pascal-inspired form without a semicolon before Else. Update the compiler if you still see this error on valid code.

expected ';' (found identifier) after a cast

C-style casts such as (Integer)c are not Vertex expressions. Use Asm for casts, or a helper function.

Asm string contains a quote

The string lexer stops at the next double quote. Do not put raw quotes inside Asm. Store text in a String variable and use .c_str() in the injected C++.

char[N] name in output.cpp

Fixed in current vertexc: local arrays emit as char name[N]; Rebuild the compiler if you still see the old form.

GUI link errors

Link with -mwindows and at least -luser32 -lgdi32. Static linking may also need -static-libgcc -static-libstdc++. vertex_build.py applies these flags in GUI mode.

ComPort not declared / Label used for serial control

Use an IDE build that maps comport to ComOpen and declares Integer variables. Prefer Generate Code after placing controls so Var and form sections stay aligned.

Import not found

The .vtx path is relative to the working directory of vertexc. Place vcl.vtx next to your program or run the compiler from the project folder.

Keywords must be capitalised

Write If, Then, Else, Run, Stop, not if, then, else. Identifiers you invent may use any casing.

End of the Vertex Language Reference. For the visual designer, form sync, and project workflow, use Vertex IDE with `vertex_build.py` and `vcl.vtx` in the same distribution.